Duplicate Question Retrieval on the Quora Dataset

Project ID: NLP-4 | Team: Blen Abate, Debosmita Chakraborty, Anusha Gairola

1. Problem Statement

Question-and-answer websites often receive the same question many times in different words, such as "How can I lose weight fast?" and "What is the quickest way to lose weight?". Finding the existing question that matches a new one saves users time and keeps answers in one place. We build a search system for Quora, a question-and-answer site, with a collection of 522,931 (about 523K) previously asked questions.

- **Input:** a new question in plain text, plus the collection of existing questions.
- **Output:** a ranked list of the top K existing questions that are duplicates of the new question.
- **Connection to ML:** word matching fails when two questions use different words for the same meaning, so we can compare keyword-based ranking against neural models that represent meaning as vectors.
- **Scope:** we run a fair comparison of a few retrieval methods and analyze where they fail. We are not aiming for a leaderboard rank.

2. Data and Resources

- **Source:** BEIR Quora Dataset. The dataset contains corpus.jsonl with approximately 523K questions, queries.jsonl containing 15K evaluation queries, and qrels/dev.tsv and qrels/test.tsv containing the relevance judgments for the development and test queries.
- **Access:** We will download the preprocessed Quora dataset from the official BEIR dataset repository.
- **Constraint:** We will use the provided dev split for development/model selection and the test split only for final evaluation. We will not collect additional Quora data or use external datasets.

3. Baseline and Planned Methods

1. **Baseline 1, TF-IDF:** represent questions using TF-IDF vectors and rank existing questions by cosine similarity.
2. **Baseline 2, BM25:** use BM25, a stronger keyword-based retrieval method, to rank the existing questions.
3. **Dense retrieval:** use pretrained sentence-embedding models (all-MiniLM-L6-v2 and the larger all-mpnet-base-v2) to represent questions as dense vectors and retrieve the most similar questions using cosine similarity. Corpus embeddings are computed once and cached to disk.
4. **Retrieve-and-rerank:** take the top 20–50 candidates returned by the best first-stage retrieval method and rerank them using a pretrained cross-encoder based on MS MARCO MiniLM.
5. **Analysis:** compare retrieval methods by preprocessing, model size, reranking depth, query length, word overlap, and retrieval time.

4. Evaluation Plan

- **Development and testing:** use the provided dev split for model development and tuning, and reserve the provided test split for final evaluation.
- **Fairness:** all methods search the same corpus, use the same query splits, the same K values, and the same metrics. A query is never allowed to retrieve itself.
- **Leakage prevention:** no fine-tuning is done, so no training data comes from dev or test queries. Pretrained models are used as released, dev queries are used for all tuning, and test queries are used once at the end. The pretrained models may have seen Quora-style duplicate pairs during their own training, which we report as a limitation.
- **Error analysis:** examine low-ranked or missed duplicates, performance by query length and word overlap, and cases where dense retrieval outperforms keyword search.

5. Team Roles and Plan

- **Roles** (each member owns one part of the pipeline and one analysis):
    - **Blen Abate:** data loading, EDA, query sampling, evaluation code, and both baselines (TF-IDF and BM25). Analysis: preprocessing effects and results by query length.
    - **Debosmita Chakraborty:** dense retrieval using MiniLM and MPNet. Analysis: compare model performance, retrieval speed, and the effect of model size.
    - **Anusha Gairola:** cross-encoder reranking, final results tables and figures, and failure analysis. Analysis: reranker depth study and qualitative examples.
    - **All:** README, report sections, presentation, and regular Git commits. We will review each other's code so every member understands the full pipeline.
- **Weeks 3-8:**
    - Weeks 3-4: set up Git, environment, and data; perform exploratory analysis(query lengths, number of duplicates per query) and fix the query samples.
    - Weeks 5-6: build evaluation code and run TF-IDF and BM25 on the dev set.
    - Weeks 7-8: implement MiniLM, run initial dense retrieval, prepare progress summary.
- **Weeks 8-13:**
    - Weeks 9-10: add MPNet and the cross-encoder; tune on the dev set.
    - Weeks 11-12: ablations, error analysis, and optional hybrid method; evaluate on test set.
    - Week 13: finalize the report, code, README, and reproducibility files.

6. Risks and Fallbacks

• **Colab limits (session timeouts, GPU availability)**. Free Colab sessions can disconnect or have no GPU at times. Fallback: save embeddings and model checkpoints to Google Drive, use the smaller embedding model, rerank only the top 20 candidates, and encode the corpus in parts, saving progress after each part. CPU-only encoding of 523K questions would take many hours, so CPU is a last resort and only for MiniLM.

• **Data size (memory and storage requirements)**. Processing 523K questions needs substantial memory. Fallback: encode and score in batches, save embeddings to disk, and use the smaller MiniLM model if needed.